
DA6400 – Assignment 2

Name: Yash Purswani, Govind S Ashan, Tiramdas Abhinand **Due Date:** April 7 2026, 11:59 PM
Student Number: ME22B214, ME23B168, ME23B208 **Assignment:** 2

Division of Work: Yash was responsible for tackling Questions 1, 2, and 3. Abhinand completed Question 4, while Govind handled Question 5.

The Google Drive link for the code can be found here.

https://drive.google.com/file/d/1W01cdtHa5zHXs56-S6_FA6Q14dr8JSmM/view?usp=drive_link

The Google Drive link for the video can be found here.

https://drive.google.com/file/d/1R8aNXwwYz850g4VIiCJGIoxykvjnBP_X/view?usp=sharing

Problem 1 Vanilla DQN Implementation

We implement vanilla DQN based on Mnih et al. (2015). The Q-network is a 2-hidden-layer MLP (64 units per layer) with ReLU activations. This balances capacity for the 2D observation space (position, velocity) and 3 discrete actions (accelerate to the left, right, and don't accelerate) without overfitting. Linear layers use Kaiming (He) uniform initialization.

Exploration is ϵ -greedy, decaying linearly from 1.0 to 0.05 over 100,000 steps. Experience is stored in a 100,000-capacity circular replay buffer, sampling mini-batches of 64 uniformly at random. The target network hard-updates every $C = 1000$ steps. We optimize using Adam (learning rate 10^{-3}) with gradient clipping (max norm 10) to ensure early stability. The discount factor is $\gamma = 0.99$, and episodes truncate at 2000 steps.

Crucially, bootstrapping targets ($r + \gamma \max_{a'} Q_{\text{target}}(s', a')$) are masked to r *only* on true termination (reaching the flag), not artificial truncation. This aligns with Gymnasium's API and prevents biased value estimation.

Problem 2 Results and Analysis

- See Figure 1 for the return vs. timesteps curve (mean $\pm$ 95% CI over 15 seeds) for vanilla DQN with truncation length 2000.

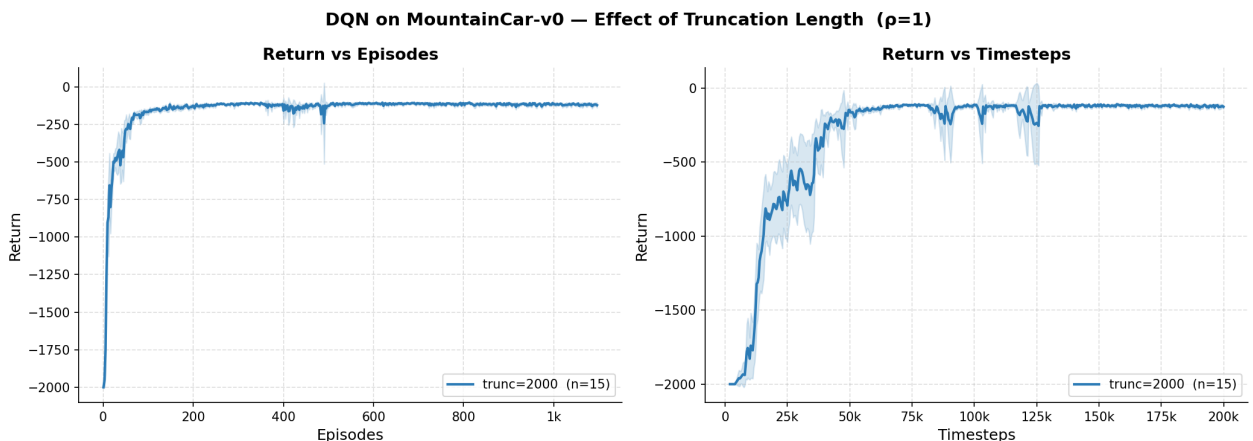


Figure 1: Vanilla DQN on MountainCar-v0 (truncation = 2000). Mean return with 95% confidence intervals over 15 seeds.

- b. **Description and interpretation:** The learning curve plateaus for the first $\sim 50\text{k}$ – 80k timesteps before sharply converging near the optimal return of -110 to -85 (an optimal policy requires ~ 85 – 110 steps).

This initial plateau reflects the sparse-reward structure (-1 per step until success). Random exploration ($\varepsilon = 1$) rarely reaches the goal, filling the replay buffer with uninformative transitions. Once a success occurs, valid Q-values propagate through the replay buffer, driving rapid improvement.

Does the car directly accelerate toward the target? No. The engine is too weak to climb the right hill directly. The optimal, non-myopic strategy is to accelerate *leftward* to build momentum on the left slope, then reverse to crest the right hill. DQN learns this delayed-reward oscillating behavior via long-range Q-value backups.

Problem 3 Effect of Episode Truncation Length

- a. See Figure 2 for a comparison of DQN with truncation lengths of 200, 1000, and 2000 timesteps.

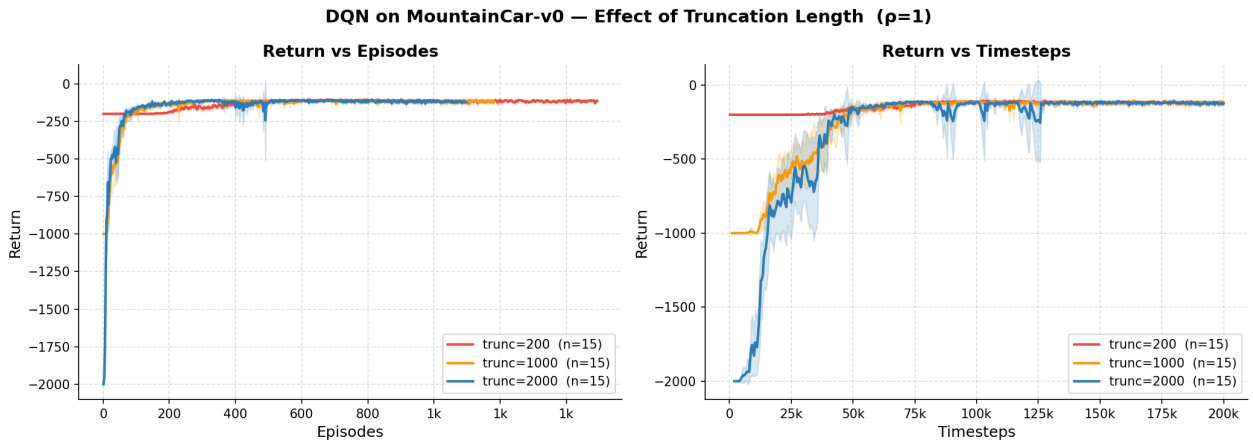


Figure 2: DQN on MountainCar-v0 with truncation lengths 200, 1000, and 2000. Mean return with 95% CI over 15 seeds.

- b. **Comparison:** Shorter truncation severely degrades learning. At 200 steps, the agent rarely succeeds within 200k timesteps. Convergence speed and reliability improve significantly at 1000 steps, and peak at 2000 steps.

Reasoning: An untrained agent relies on random exploration to reach the goal.

A 200-step limit prematurely resets the environment before the agent can accidentally succeed, starving the replay buffer of rewarding transitions. Longer horizons increase the probability of randomly reaching the goal and initiating the learning signal.

- c. **Why were episode truncations introduced in Gymnasium?**

The original unbounded formulation is problematic for benchmarking. Without limits: 1) Untrained agents might run indefinitely. 2) Fair algorithmic comparison on a fixed computational budget becomes impossible. 3) Replay buffers would fill with endlessly long, failed trajectories. Truncation ensures experiments remain tractable and reproducible.

- d. **How can truncation be harmful, and what is a good value?**

Truncation introduces two main risks:

- *Value function bias:* Incorrectly treating truncation as termination (bootstrapping to r instead of $r + \gamma \max_{a'} Q(s', a')$) artificially penalizes states near the goal. We avoid this by isolating true terminations.

- *Exploration starvation*: Too-short horizons prevent random discovery of the sparse reward. Since an optimal policy takes $\sim 85\text{--}110$ steps, a 200-step limit is too restrictive for an untrained agent. A limit of **1000–2000 steps** provides sufficient room for random exploration while maintaining computational efficiency, fully justifying the assignment’s choice of 2000.

Problem 4 DQN Implementation with replay factor

a. Experiments with different replay factor (ρ)

We evaluated the DQN agent using varying replay factors ($\rho \in \{1, 2, 4, 8\}$). The replay factor controls the number of gradient descent updates (mini-batch samples) performed per single environment step. The results are analyzed below based on the generated learning curves (Return vs. Episodes).

- i. **Comparison of results:** As seen in the learning curve plot 3, all configurations successfully learn the MountainCar task, escaping the initial -2000 plateau between 20 and 50 episodes and converging to an asymptotic return of approximately -110 to -150 . However, the stability of the training varies wildly. The baseline $\rho = 1$ (blue) is the most stable. As ρ increases to 4 (green) and 8 (red), the variance increases drastically, with $\rho = 8$ showing severe catastrophic drops in performance late in training (e.g., around episodes 550–600).
- ii. **Statistical significance of differences:** During the initial learning phase (the steep ascent), the 95% confidence intervals of all ρ variants overlap almost entirely. Therefore, the differences in learning speed (sample efficiency in terms of episodes) are **not statistically significant**. However, the late-stage instability of $\rho = 8$ is likely statistically significant, as its variance spikes extend far below the tight confidence band of $\rho = 1$.
- iii. **Why differences exist (Effect of increasing ρ):** Increasing the replay factor means the Q-network is updated much more aggressively relative to the intake of fresh environmental experience.
 - *Learning/Sample Efficiency*: Theoretically, higher ρ should increase sample efficiency by extracting more information from rare successful trajectories. However, in MountainCar, the bottleneck is *discovering* the goal. Once discovered, all variants learn quickly, which is why sample efficiency looks similar across ρ values here.
 - *Final Policy Performance & Stability*: High ρ leads to overfitting on the current distribution of the replay buffer. Updating too many times on highly correlated, stale data causes the Q-values to oscillate, destabilizing the target network. This explains why $\rho = 8$ suffers from severe performance collapse while $\rho = 1$ maintains a stable optimal policy.
- iv. **Deployment advantages, disadvantages, and real-world scenario:**
 - *Advantageous*: Increasing ρ is highly beneficial when environment interactions are extremely expensive, slow, or dangerous, but computation is cheap.
 - *Disadvantageous*: It is detrimental when safety and stability are strictly required, or when computational resources (wall-clock time/FLOPs) are the primary bottleneck.
 - *Real-world scenario*: Consider training an **autonomous Mars Rover**. Physical trial-and-error is incredibly slow and risky (high sample cost). Here, a higher ρ is advantageous because the rover can take one physical step, and then use its onboard computers to perform multiple updates (“think”) on its past data before risking the next move. However, the engineers must carefully tune ρ to avoid the instability seen in our $\rho = 8$ results, otherwise, the rover’s policy might suddenly degrade, leading to a catastrophic physical crash.
- v. **Relation to Planning:** Planning relies on using a model of the environment to improve

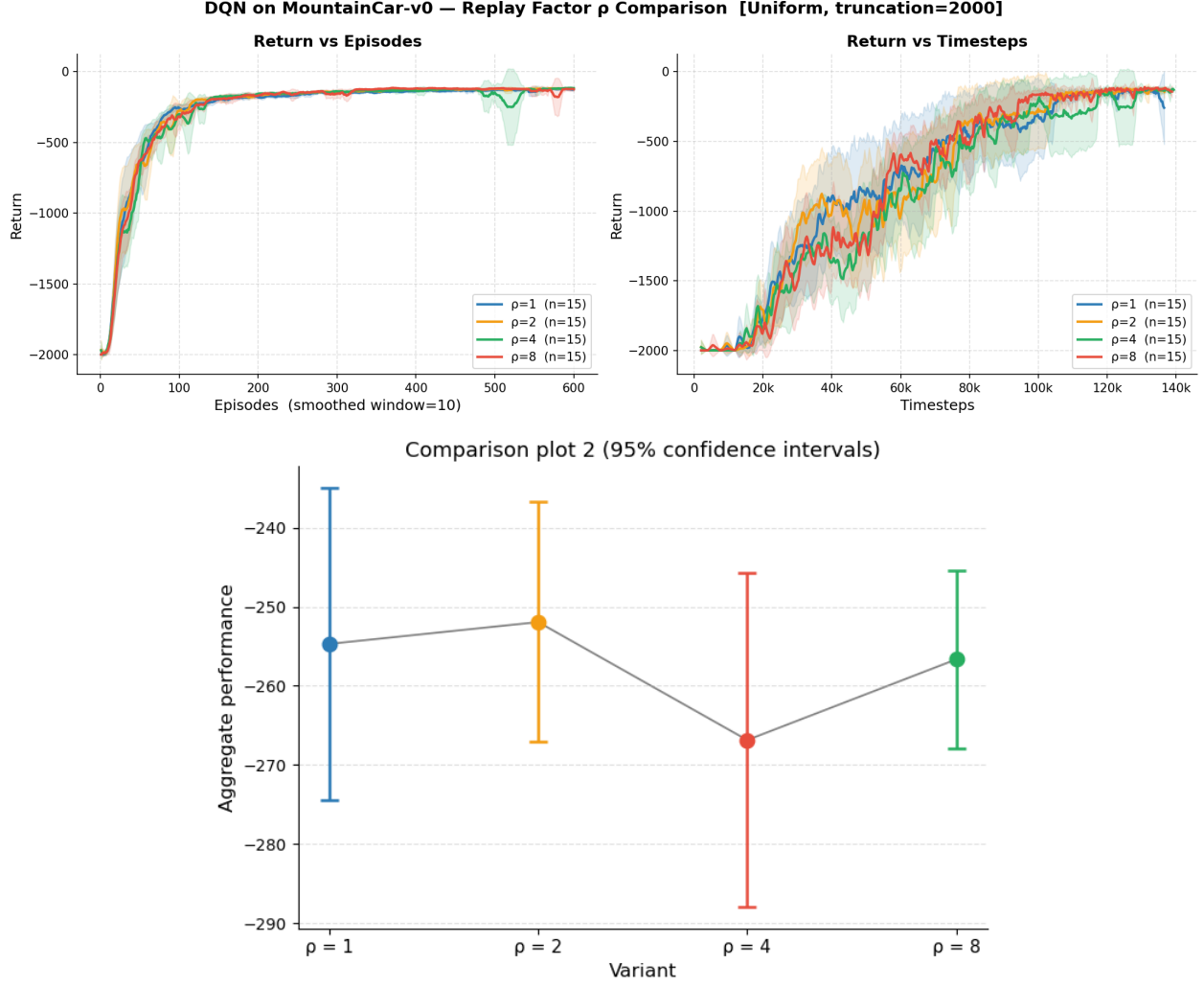


Figure 3: Replay factor comparison for $\rho \in \{1, 2, 4, 8\}$. Mean performance with 95 % confidence intervals over 15 seeds

policies without taking real actions. In DQN, the replay buffer acts as a *data-driven, empirical model* of the environment (storing samples of $P(s'|s, a)$ and $R(s, a)$). When $\rho > 1$, the agent takes one real environment step, but then performs multiple ”simulated” steps by sampling from this empirical model to update its value function. This is fundamentally a background planning process, conceptually identical to the **Dyna architecture** (e.g., Dyna-Q), where real experience is used simultaneously to learn the policy and to ”plan” via simulated experience.

b. **Distribution of Performance** We analyze the aggregate performance distributions (smoothed using Kernel Density Estimation) to understand the spread and modalities of the agents’ returns across different replay factors.

- i. See Figure 4 for aggregate performance distribution for different replay factors $\rho \in \{1, 2, 4, 8\}$.
- ii. **Distribution Characteristics:** All distributions exhibit clear **multimodality** and are strongly left-skewed, deviating significantly from a normal distribution. Even for lower replay factors ($\rho = 1, 2$), multiple peaks are visible, indicating distinct performance regimes across different training runs.

As ρ increases ($\rho = 4, 8$), this multimodality becomes more pronounced. In addition to

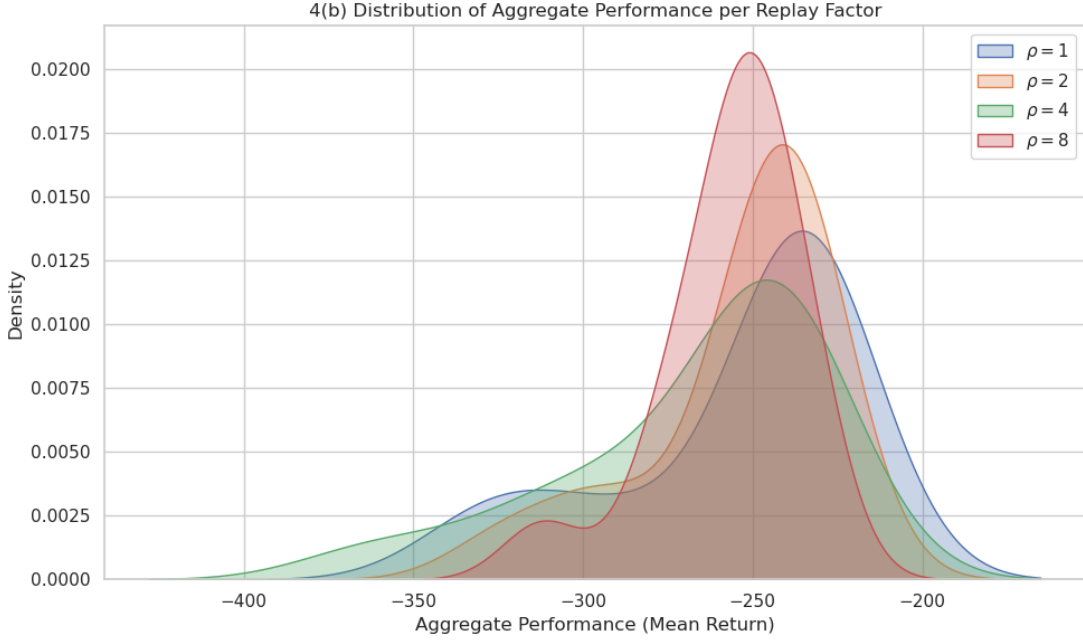


Figure 4: Distribution of aggregate performance (mean return) for different replay factors ρ .

a dominant peak around -240 to -260 , secondary modes and heavier left tails emerge (extending toward -300 to -350). These additional modes correspond to runs where learning becomes unstable or converges to suboptimal policies.

Overall, increasing ρ does not simply widen the distribution, it reshapes it into multiple distinct outcome clusters, reflecting fundamentally different training trajectories rather than mere variance.

- iii. **Nuanced Inferences:** While the learning curves in Part (a) suggested similar mean final performance across all ρ , the distribution plot reveals structural risks hidden by averages. The presence of multiple modes across all ρ values indicates that DQN training is inherently stochastic and can converge to qualitatively different policies. Importantly, higher replay factors ($\rho = 4, 8$) amplify the probability mass in poorer-performing modes, showing that increasing ρ not only increases variance but also increases the likelihood of catastrophic training outcomes.
- iv. **Beyond DQN-MC (Why distributions matter):** Visualizing performance distributions is critical in safety-critical deployments (e.g., autonomous driving, medical robotics). A standard learning curve might show a high average reward while masking a multimodal distribution where a non-trivial fraction of runs result in catastrophic failure. Distributional analysis exposes these worst-case tail risks and distinct failure modes, providing a far more realistic assessment of reliability than mean-based metrics alone.

c. **Variability in Performance** Tolerance intervals provide bounds within which a specified proportion of individual episode returns are expected to fall, offering a stricter view of reliability than confidence intervals.

- i. See Figure 5 for performance as a function of replay factors $\rho \in \{1, 2, 4, 8\}$.
- ii. **Comparison of Intervals:** The information here is drastically different from Part (a). While the 95% confidence intervals in Part (a) were narrow (tightly bounding the *mean*), the 90% tolerance intervals here are massive, spanning thousands of units. Even late in training (episodes 400–600), the tolerance intervals for $\rho = 4$ and $\rho = 8$ spike violently

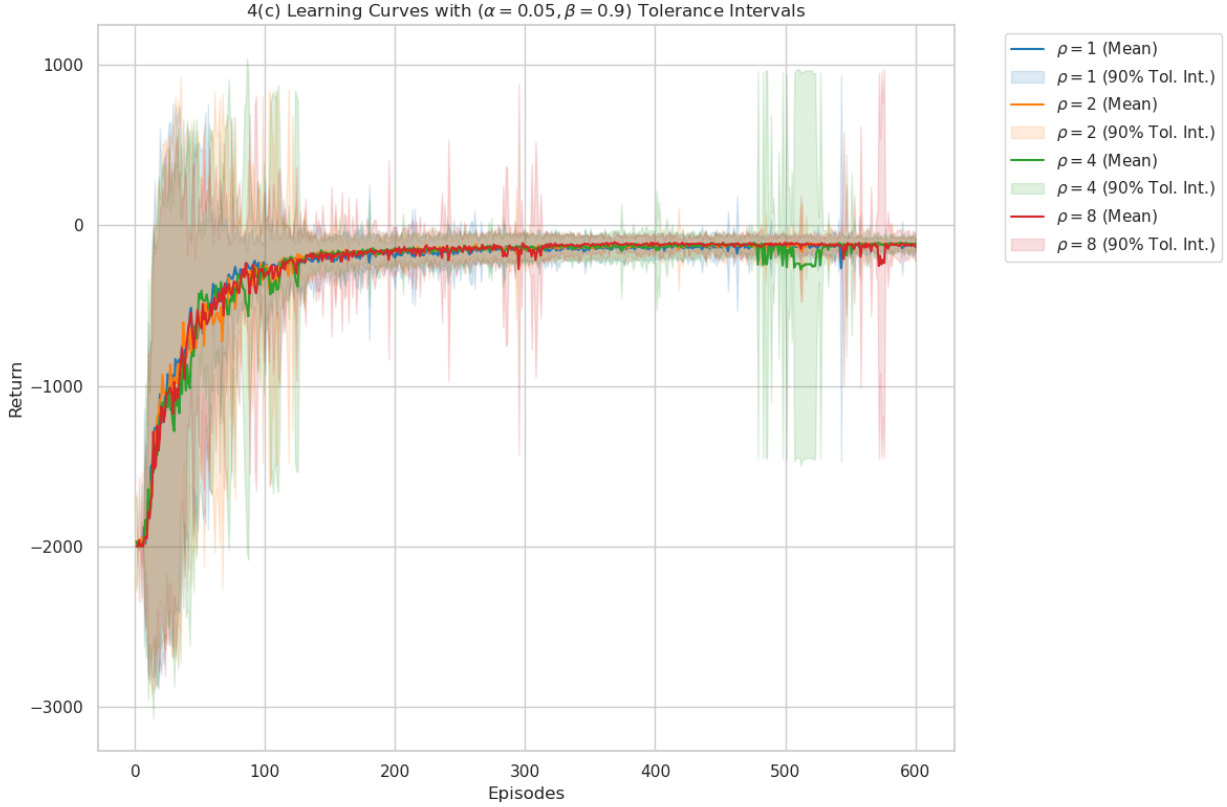


Figure 5: Learning Curves with $(\alpha = 0.05, \beta = 0.9)$ Tolerance Intervals for varying replay factors.

downward to -1500 and upward. This shows that individual episode performance is highly erratic, a detail completely masked by the smooth mean curves in Part (a).

- iii. **Reliability and Worst-case Performance:** The system is fundamentally unreliable on an episode-to-episode basis, especially at higher replay factors. While the "average" agent converges to -110 , the worst-case performance bounds indicate that an agent trained with $\rho = 8$ or $\rho = 4$ can randomly experience total policy collapse during deployment, scoring worse than -1000 on a given episode. $\rho = 1$ is the most robust, though it still exhibits noticeable per-episode variance.

iv. **Tolerance vs. Confidence Intervals:**

- *Fundamental Difference:* A Confidence Interval (CI) bounds a population parameter (e.g., "We are 95% confident the *true average return* is in this range"). A Tolerance Interval (TI) bounds the actual sample distribution (e.g., "We are 95% confident that *90% of all individual episode returns* fall in this range").
- *Usefulness:* CI is useful for claiming an algorithm is statistically "better on average". It is highly misleading for safety guarantees. TI is useful for worst-case bounds and deployment safety.
- *Convergence:* As the number of runs $N \rightarrow \infty$, the Confidence Interval shrinks to a width of zero (converging exactly on the true mean). The Tolerance Interval does *not* shrink to zero; it converges to the actual statistical spread (the true 90% variance) of the underlying data distribution.

d. **Sensitivity Analysis of Hyperparameters**

To understand the robustness of DQN, we analyse sensitivity to **mini-batch size** and **target network update frequency**, across $\rho \in \{1, 4\}$. Performance is measured as the mean return

over the last 100 episodes, averaged across 15 seeds.

i. Mini-batch size

See Figure 6 for performance as a function of batch size $B \in \{16, 32, 64, 128, 256\}$.

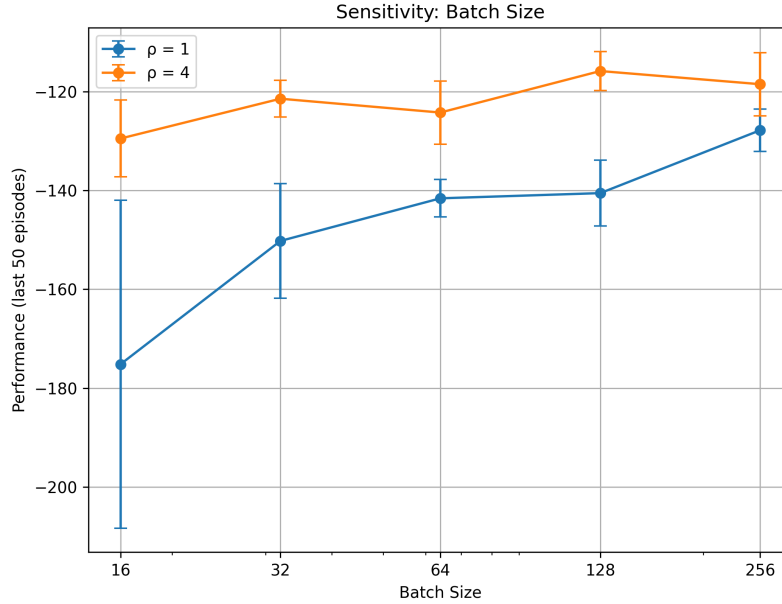


Figure 6: Sensitivity to mini-batch size for $\rho \in \{1, 4\}$. Mean performance with 95% confidence intervals over 15 seeds.

Description and interpretation

Performance improves significantly when increasing B from 16 to 64, after which gains saturate. Very small batch sizes ($B = 16$) lead to noisy gradient estimates and unstable learning, particularly for $\rho = 1$. As B increases, gradient estimates become more reliable, improving convergence.

However, very large batches ($B = 256$) do not yield further gains and can slightly degrade performance. This is likely due to fewer parameter updates per unit interaction with the environment, reducing learning efficiency.

Across all batch sizes, $\rho = 4$ consistently outperforms $\rho = 1$, indicating that performing multiple updates per interaction improves sample efficiency.

Given that the total number of samples processed per step is $\rho \times B$, this introduces a trade-off:

- Larger B with smaller ρ leads to stable but less frequent updates,
- Smaller B with larger ρ leads to more frequent but noisier updates.

From the results, a **moderate batch size (64–128) combined with higher ρ** provides the best balance between stability and learning speed.

ii. Target network refresh rate

See Figure 7 for performance as a function of target update frequency $C \in \{250, 500, 1000, 2000, 4000\}$.

Description and interpretation

The target network stabilizes temporal-difference updates by providing a slowly changing reference. Performance is highest for intermediate update frequencies ($C \approx 500$ – 1000).

When the update frequency is too high (e.g., $C = 125$), the target network changes too rapidly, reducing its stabilizing effect and reintroducing the moving target problem. This leads to increased variance and less stable learning.

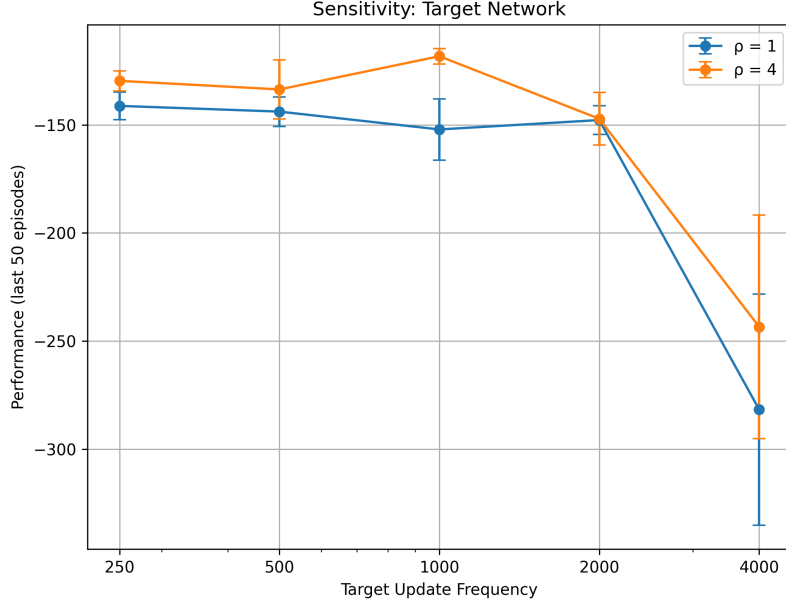


Figure 7: Sensitivity to target network update frequency for $\rho \in \{1, 4\}$. Mean performance with 95% confidence intervals over 15 seeds.

Conversely, when the update frequency is too low (e.g., $C = 4000$), the target network becomes stale. This delays the propagation of newly observed rewards, significantly degrading performance.

Comparing $\rho = 1$ and $\rho = 4$, increasing ρ improves performance when C is well-chosen, but does not compensate for poor choices of C . In particular, for very large C , $\rho = 4$ can exacerbate instability by repeatedly updating towards outdated targets.

Overall, the results suggest that **moderate target update frequencies** ($C = 500\text{--}1000$) provide the best trade-off between stability and learning speed.

iii. Deployment Lessons for the Mars Rover Scenario

In the Mars Rover scenario, utilizing a higher replay factor (ρ) is essential to maximize learning from scarce, high-risk physical interactions. However, our sensitivity experiments highlight that DQN’s deployment robustness is highly fragile and demands strict hyperparameter co-tuning:

- **Batch Size (B) Trade-offs:** While a high ρ improves sample efficiency, pairing it with a small batch size ($B = 16$) to save onboard compute causes noisy, erratic gradient updates. For the rover, this translates to unpredictable, unsafe physical manoeuvres. A moderate B (64–128) is strictly required to anchor the frequent updates with reliable gradients.
- **Target Network Staleness (C):** A high ρ acts as an amplifier for poor target network tuning. If C is too large ($C = 4000$), the target becomes stale. The rover would perform rapid, repeated updates reinforcing outdated, potentially dangerous logic before the network can correct itself based on new terrain.
- **Reliability Lesson:** DQN lacks inherent safety bounds. You cannot alter one parameter for operational efficiency (like increasing ρ to save physical steps) without carefully balancing stabilization mechanisms (B and C). Poor combinations lead to sudden, catastrophic failure rather than graceful degradation, making offline co-tuning critical

before real-world deployment.

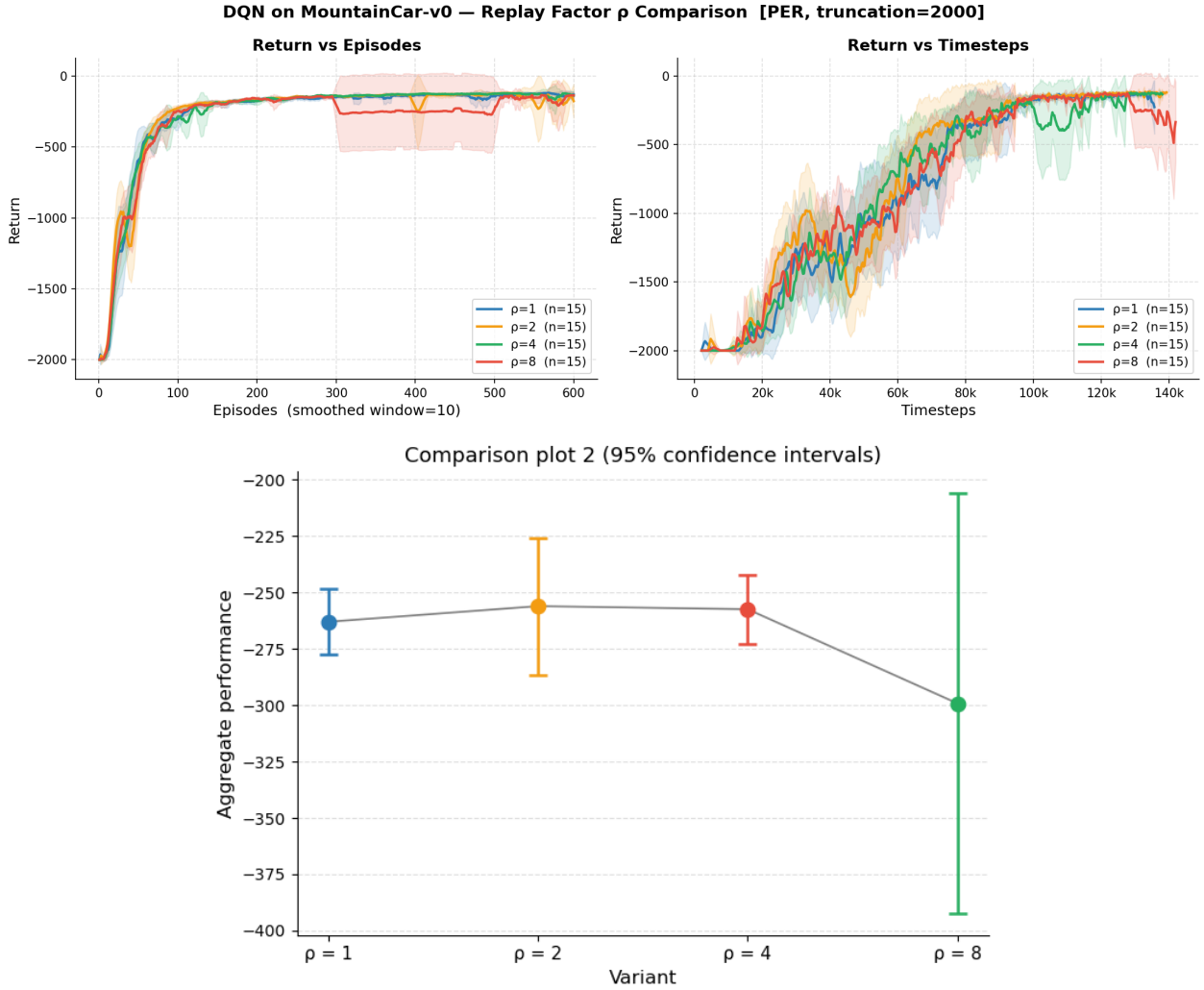


Figure 8: Comparison of DQN with Uniform Sampling vs. Prioritized Experience Replay (PER) across different replay factors. Mean performance with 95% confidence intervals over 15 seeds.

Problem 5 (Bonus) Effect of PER on Replay Factor (ρ):

Using Prioritized Experience Replay (PER) **elevates the negative effects** of increasing the replay factor. While increasing ρ under uniform sampling introduces moderate instability due to aggressive data reuse, pairing a high ρ with PER drastically destabilizes training, often leading to divergence or severe performance collapse.

Why does this happen?

1. **Catastrophic Overfitting to Narrow Data:** PER skews sampling toward transitions with the highest Temporal Difference (TD) error. When $\rho > 1$, the agent performs multiple gradient updates before collecting new data. Under PER, these consecutive updates will repeatedly draw the exact same "surprising" transitions. Instead of generalizing, the network severely overfits to this small, biased subset of the state-action space.
2. **Loss of Batch Diversity:** Uniform sampling with high ρ still guarantees that diverse experiences are uniformly drawn from the buffer. PER effectively shrinks the "active" size of the buffer to only the highest-priority samples. This starves the network of the diverse, well-learned (low TD

error) transitions required to maintain a stable, globally accurate value function.

3. **Amplification of Bias and Churn:** PER inherently introduces sampling bias. Even with Importance Sampling (IS) weights to correct this, performing multiple rapid updates ($\rho > 1$) on high-error samples causes the Q-values to swing violently. By the time the target network updates, the value landscape is highly distorted, creating a feedback loop of erratic TD errors and unstable learning.

In summary, PER concentrates updates on a narrow distribution of data. Multiplying those updates via a high replay factor acts as a destructive multiplier on that concentration, breaking the stability mechanisms that make experience replay effective.